Week 2

Constraint Propagation

◎ Learning Objectives

By the end of this chapter, you will be able to:

- **(H1)** define node consistency, (binary) arc consistency, and generalized arc consistency (GAC) formally, and determine by hand whether a small CSP – including one built from a global `AllDifferent` constraint – is consistent at each level;
- **(H2)** trace the AC-3 algorithm by hand on a small binary CSP (worklist of arcs, the `Revise` step, domain changes), explain why the algorithm is guaranteed to terminate at a fixpoint, and state its textbook worst-case time complexity;
- **(H3)** explain, for each of `AllDifferent`, `Element`, `Table`, `Circuit` and `Cumulative`, what the constraint propagates (which values it can remove and on what basis), and why a global constraint's filtering can be strictly stronger than an equivalent set of decomposed binary constraints;
- **(H4)** build two CP-SAT models of the same problem – one using a global constraint, one using an equivalent pairwise decomposition – run both, and read the solver's own search statistics (`NumBranches`, `NumConflicts`, `WallTime`) to compare their propagation strength empirically, on real captured output;
- **(H5)** connect an empirically observed propagation-strength difference (H4) back to the consistency-level argument that predicts it (H1, H3) – closing the loop from "why a global constraint should prune more" to "here is the solver's own evidence that it did (or did not, and why that is still informative)."

Last week deliberately treated `solver.Solve(model)` as a black box that simply hands back a solution. This week opens that box and shows *how* a solver narrows the search before it ever branches on a variable's value. Four units carry this chapter, in a fixed order: local consistency at three increasing strengths (node, arc, generalized arc), the AC-3 algorithm that actually achieves arc consistency and the fixpoint it reaches, what this course's five key global constraints propagate and why, and, as this week's payoff, measuring propagation strength empirically by building two CP-SAT models of the *same* problem and comparing the solver's own statistics between them.

This chapter stays deliberately out of several topics later weeks own outright. It says nothing about search-tree mechanics or variable/value ordering heuristics – no MRV, degree, dom/wdeg or LCV – since propagation here is treated purely as domain filtering that happens *before* or *between* search steps; Week 3 opens the search box itself. It introduces no local search, restarts, nogood learning or solver portfolios (Week 4), no symmetry breaking or tree-decomposition theory (Week 5), and no MiniZinc or Gurobi – every model this week is built in OR-Tools CP-SAT, continuing Week 1's tool choice, with no optimization objective anywhere: every problem here still asks only whether an assignment exists, never for the best one. Two of this chapter's five global constraints, `Circuit` and `Cumulative`, appear only as tiny, self-contained propagation demonstrations – a 4-node toy graph, three tasks on one resource – not as full routing or scheduling models; those are Week 14's and Week 11's jobs respectively.

This week also continues the problem pool from last week rather than starting over: this chapter's N-queens and map-coloring code starts directly from `kod_ornekleri/hafta01/modeller/nqueens.py` and `map_coloring.py`, carrying the variable declarations and diagonal transforms over unchanged, and varying only the constraint layer – a global `AllDifferent` against a pairwise decomposition – because that variation is exactly what this week's propagation-strength comparison needs. Sudoku is not reused this week; its `AllDifferent`-per-row/column/box structure already illustrates the point Unit 4 needs via N-queens, and a second full example would dilute rather than sharpen the comparison.

2.1 Local Consistency: Node, Arc and Generalized Arc Consistency

1 • Pre-instruction **Why It Matters**

A course-registration system lists a time slot as "still available" right up until a student tries to select it and is told every course that could use it is already full – the system was checking each field's own rules, but never checking whether a *combination* the student was heading toward could still work. That gap between "looks fine in isolation" and "actually works together" is precisely what this unit's three levels of consistency measure, and it is the first thing a real solver checks before it does anything as expensive as search.

1 • Pre-instruction **Prior Knowledge**

Week 1 Unit 1 gave you the formal CSP triple – variables, domains, constraints – and Week 1 Unit 4 had you call `model.AddAllDifferent(…)` in `nqueens.py` without ever explaining *why* that call helped a solver find a solution faster than a pile of separate `!=` constraints would have. This unit gives that filtering effect a name.

It is tempting to think a solver either "finds a solution" or "doesn't," with nothing interesting happening in between. In fact, before a solver commits to guessing a single variable's value, it usually spends time throwing out values that could never appear in any solution at all – a cheap, mechanical check that shrinks the problem before the expensive part even starts. That check comes in three strengths, each consulting more of the problem than the last, and this unit walks through all three in

order: node consistency, (binary) arc consistency, and generalized arc consistency.

2.1.1 Node consistency: checking a variable against itself

node consistency: A variable is node-consistent if every value remaining in its domain satisfies every unary constraint stated on that variable alone.

The weakest and cheapest kind of consistency asks nothing of any other variable. A variable has **node consistency** if every value remaining in its domain satisfies every *unary* constraint – a constraint that mentions only that one variable, with no other variable involved at all.

Definition: Node consistency

A variable is node-consistent if every value in its domain satisfies every unary constraint on that variable alone. A CSP is node-consistent if every variable is.

Consider a small scheduling variable `Slot`, with domain {9am, 11am, 2pm}, under the unary constraint "not 2pm" (the room is booked at that time). Checking node consistency here means looking only at `Slot` and its own rule: 9am satisfies "not 2pm," 11am satisfies it, and 2pm does not – so 2pm is removed, leaving {9am, 11am}. Nothing about any other variable was consulted; the removal followed entirely from `Slot`'s own domain and its own rule. This is the cheapest possible filtering step a solver can perform, and it is usually the very first one applied, before any variable is compared against any other.

Analogy

Think of a "preferred campus" dropdown menu that never lists a campus already announced as closed this term. That dropdown was made node-consistent before you ever opened the form – no other field on the form was consulted, only that one field's own rule about which campuses still count as valid choices.

A common misunderstanding at this point is treating node consistency as though it says something about how a variable relates to *other* variables. It does not – by definition, a unary constraint mentions only one variable, so node consistency can never, by itself, detect a problem that only shows up once two or more variables are considered together. That is exactly the gap the next level closes.

2.1.2 (Binary) arc consistency: checking a variable against one neighbour

support: A value v for X_j supports a value u for X_i under a binary constraint if the pair (u, v) together satisfies that constraint.

Once every variable's domain has been trimmed by its own unary rules, the next question is harder: for a pair of variables linked by a constraint, does every value remaining in one variable's domain still have some partner value in the other variable's domain that keeps the constraint satisfied? A value v for a variable X_j is said to give **support** to a value u for a variable X_i under a binary constraint if the pair (u, v) jointly satisfies that constraint.

Definition: Support

A value v for X_j supports a value u for X_i under a binary constraint if the pair (u, v) satisfies the constraint.

(binary) arc consistency: An arc (X_i, X_j) is consistent if every value remaining in X_i's domain has at least one supporting value in X_j's current domain; a CSP is arc-consistent if this holds for every arc.

An arc (X_i, X_j) has **(binary) arc consistency** if every value still in X_i's domain has at least one supporting value in X_j's *current* domain – current, because a value's support can itself disappear once X_j loses values of its own. A whole CSP is arc-consistent if this holds for every arc in the problem. Any value of X_i with no support at all in X_j can safely be removed: no solution could ever have used it, since it could never be paired with anything acceptable in X_j.

Definition: (Binary) arc consistency

An arc (X_i, X_j) is consistent if every value remaining in X_i's domain has at least one supporting value in X_j's current domain; a CSP is arc-consistent if this holds for every arc.

Analogy

Picture two students cross-checking a shared exam-room sign-up sheet before either commits to a slot. Each asks: "for every time still on this list, is there still a time on the other list that does not clash with it?" A time with no such partner left on the other student's list gets crossed off – exactly what removing an unsupported value does.

Notice the phrase "current domain" in the definition above is doing real work. Removing an unsupported value from X_i can, in turn, remove the very support that some value of a *third* variable was relying on – which is exactly why checking arc consistency is not a single pass over the problem's arcs, but a process that has to revisit arcs whose neighbours have changed. Unit 2 turns that observation into an actual algorithm; for now, hold onto the idea that one removal can trigger another, several arcs away from where it started.

2.1.3 Generalized arc consistency: checking a variable against a whole constraint's group

generalized arc consistency (GAC): The arc-consistency idea lifted to a constraint of any arity: a value for one variable in the constraint's scope is GAC-consistent if it extends to a complete assignment of every variable in that scope satisfying the constraint.

Binary arc consistency only makes sense for constraints between exactly two variables. Many of the constraints this course uses – starting with `AllDifferent` in Week 1 – involve three, four, or dozens of variables at once. The natural generalization carries the name **generalized arc consistency (GAC)**: a value for a variable X_i that appears in a constraint C's scope is GAC-consistent with C if that value can be extended to a complete assignment of *every* variable in C's scope that satisfies C.

Definition: Generalized arc consistency (GAC)

A value for one variable in a constraint's scope is GAC-consistent if it extends to a complete assignment of every variable in that scope satisfying the constraint. A CSP is GAC if this holds for every value of every variable, with respect to every constraint on it.

The key word is *complete*: binary arc consistency only ever checks one other variable at a time, but GAC insists that a value survive a check against the constraint's *entire* group of variables, all at once. That difference is not a technicality – it can catch infeasibility that a purely pairwise check misses entirely, as the next worked example shows.

Case Study 2.1: GAC catches what pairwise checking misses

Three variables A, B, C each have domain $\{1, 2\}$, under the single constraint `AllDifferent(A, B, C)`. Check $A = 1$ against B alone: it has a supporting value, $B = 2$ (the pair $A = 1, B = 2$ satisfies $A \neq B$). Check $A = 1$ against C alone: it has a supporting value too, $C = 2$. Every value of every variable passes this kind of check exactly the same way – individually, against each other single variable in turn, nothing ever looks wrong.

Now ask the GAC question instead: can $A = 1$ be extended to a *complete* assignment of A, B *and* C together that satisfies `AllDifferent`? With only two values, $\{1, 2\}$, available for three variables that must all differ, the answer is no – by the pigeonhole principle, three variables cannot take three pairwise-distinct values from a pool of only two. This is true for *every* value of *every* variable here, so a GAC-aware check on the global `AllDifferent` constraint removes every remaining value from every domain at once – a **domain wipeout** – correctly and immediately reporting that this CSP has no solution. Checking the three pairwise constraints $A \neq B$, $A \neq C$, $B \neq C$ one arc at a time, by contrast, never finds this wipeout: each pair, considered alone, is still perfectly satisfiable.

domain wipeout: The event of a variable's domain becoming empty during propagation, proving the CSP (or the current partial assignment) has no solution.

Definition: Domain wipeout

The event of a variable's domain becoming empty during propagation, proving the CSP (or the current partial assignment) has no solution.

Analogy

This is the same gap a hiring committee runs into comparing a group interview schedule against checking candidates pairwise: asking "can candidate A and candidate B both still get a distinct interview slot" for every pair, one pair at a time, can look fine for every single pair – and yet there might be no way to give *all* candidates distinct slots at once if there are more candidates than slots. Only checking the whole group together, the way a global constraint does, catches that.

Common Pitfall

A frequent mistake at this point is assuming that "every pairwise check passed" means "the whole group is fine." It does not: pairwise arc consistency only ever asks, one relationship at a time, whether a value has *some* compatible partner in one other variable. It never asks whether a single joint assignment satisfying every relationship simultaneously actually exists across the whole group – and the pigeonhole example above shows that these two questions can have different answers. Unit 3 names the algorithm (Régin's matching-based filtering) that lets `AddAllDifferent` achieve true GAC in practice, and Unit 4 measures the resulting gap on a real solver.

2.1.4 The ladder: node, arc, and GAC together

Node consistency, arc consistency, and GAC form a ladder of *how much of a variable's neighbourhood gets consulted* before a value is trusted. Node consistency asks nothing of anyone else – it checks a variable purely against its own unary rules. Arc

consistency asks one other variable at a time, and only for binary constraints. GAC asks an entire constraint's group of variables jointly, whatever that constraint's arity happens to be. Climbing this ladder costs more computation per check, since checking a whole group jointly is harder than checking one pair – but a stronger level of consistency can remove dead values, like the ones the pigeonhole example wiped out, that a weaker level would leave sitting in a domain until search stumbled onto them the hard way.

3 • Participation **Now You Try**

Three exam slots are represented by variables P, Q, R, each originally with domain $\{1, 2, 3\}$, under the constraint `AllDifferent(P, Q, R)`. This week, slot 3 was cancelled, so every domain is now $\{1, 2\}$. Is this CSP generalized-arc-consistent? Check by asking, for each value of each variable, whether it has a support that also respects `AllDifferent` for the other two.

Feedback. No – this is exactly the A, B, C pigeonhole example from this unit, with the variables renamed. Three variables, two values each, and no assignment can make all three pairwise-different at once, so *no* value in *any* domain has a valid GAC support. A GAC-aware check on the global `AllDifferent` constraint empties every domain – correctly reporting that cancelling slot 3 made the three exams impossible to schedule without a clash – immediately. Checking $P \neq Q$, $P \neq R$, $Q \neq R$ one pair at a time never finds this wipeout, because each pair alone is still satisfiable.

Check Yourself 2.1

A course-registration field "Preferred Campus" (domain {North, South, Downtown}) has the unary rule "Downtown is closed this term," and separately must differ from a friend's "Preferred Campus" field. After removing Downtown, is the two-field system automatically arc-consistent? Explain what additional check arc consistency requires beyond the unary filtering.

5 • Follow-through **Transfer — Where Else?**

Every `solver.Solve(model)` call you make for the rest of this course runs some form of this consistency-checking silently, before and during search – Week 3 shows what happens once search starts consulting these same domains to decide which variable to branch on next, and Week 11's scheduling models rely on exactly this "detect the impossible combination before committing to it" idea to keep resource-constrained problems tractable.

2.2 AC-3 and the Propagation Fixpoint

1 • Pre-instruction **Why It Matters**

Watching a solver's internal log scroll past looks like magic the first time – thousands of domain changes in milliseconds – but every one of those changes is the same three-line check from this unit, run over and over on a worklist. Once you have traced that check by hand once, the log stops being magic and starts being readable.

1 • Pre-instruction **Prior Knowledge**

Unit 1 defined (binary) arc consistency and support as a static property: an arc either is consistent or it is not. This unit turns that definition into an algorithm that *achieves* arc consistency, and Week 1's discrete-mathematics prerequisite – sets, relations – is what makes "domain as a shrinking finite set" a familiar picture rather than a new idea.

Unit 1 left one loose thread deliberately untied: removing an unsupported value from one variable's domain can remove the very support another variable's value was relying on, several arcs away from where the first removal happened. Checking every arc exactly once, in a single pass, is therefore not enough to guarantee arc consistency – a change discovered late in the pass can invalidate an arc that was already checked earlier. This unit's algorithm, **AC-3**, solves that problem by keeping a running list of arcs still owed a check, and adding arcs back to that list whenever a domain change could have invalidated them.

AC-3: The classic arc-consistency algorithm: repeatedly Revise arcs from a worklist, re-queueing affected arcs, until the worklist is empty.

2.2.1 The algorithm: a worklist and a Revise step

AC-3's shape is easy to state and only slightly harder to get exactly right: repeatedly pick an arc that still needs checking, check it, and if that check changed anything, make sure every arc it could have affected gets checked again later. Two pieces make that shape precise – a running list of arcs still owed a check, and the single check applied to each one – and this subsection defines both before working through a full trace by hand.

worklist (queue): Arcs still pending a Revise check; an arc is re-added whenever a domain change could invalidate a previously-checked arc.

AC-3 keeps track of every arc still owed a check in a single **worklist (queue)**, initialized with every arc in the problem before the algorithm's very first step. Arcs come out of this worklist one at a time, in whatever order they were added, and the worklist keeps growing again as the algorithm runs – it is not a fixed, one-pass list, and understanding why it grows back is most of this subsection's point.

Each time an arc (X_i, X_j) comes off the worklist, AC-3 runs the step called **Revise** on it: remove every value of X_i that has no supporting value left in X_j's current domain. If `Revise` removed anything at all from X_i, then every arc (X_k, X_i) for a neighbour X_k of X_i (other than X_j itself) is added back to the worklist – a value that used to support some value of X_k may have just disappeared from X_i's domain, and there is no way to know without re-checking that arc again. The algorithm keeps popping and revising arcs until the worklist is finally empty, with nothing left to re-check anywhere in the problem.

Revise: The AC-3 step that removes every value of X_i lacking a support in X_j's current domain for a given binary constraint.

Tip

The one detail students most often skip when first implementing AC-3 is re-queueing a neighbour's arcs after a successful `Revise` call. Skipping that step does not crash anything – it silently produces a result that *looks* finished but is not actually arc-consistent, because some arc that should have been re-checked after a later change never was. If a hand trace or a program's output leaves a value behind that a fully worked fixpoint would have removed, re-queueing is almost always the missing step.

2.2.2 A worked trace: the chain $X < Y < Z$

Take the chain CSP $X < Y$, $Y < Z$, with all three domains starting at $\{1, 2, 3\}$. Table 2.1 lists the worklist processing in order.

Table 2.1: AC-3 on the chain CSP $X < Y < Z$, domains starting at $\{1, 2, 3\}$.

Arc	Domain change	Re-queued because of the change
(X, Y)	removes $X = 3$ (no $y > 3$ in Y); $X = \{1, 2\}$	nothing: X's only neighbour is Y
(Y, X)	removes $Y = 1$ (no $x < 1$ in X); $Y = \{2, 3\}$	(Z, Y): Z is Y's other neighbour
(Y, Z)	removes $Y = 3$ (no $z > 3$ in Z); $Y = \{2\}$	(X, Y): X is Y's other neighbour
(Z, Y)	removes $Z = 1, Z = 2$ (no $y < z$ with $Y = \{2\}$); $Z = \{3\}$	nothing: Z's only neighbour is Y
(Z, Y)	no further change (duplicate entry)	–
(X, Y)	removes $X = 2$ (no $y > 2$ left); $X = \{1\}$	nothing

Reading the table top to bottom, one revision at a time: the worklist starts as $(X, Y), (Y, X), (Y, Z), (Z, Y)$. Revising (X, Y) first removes $X = 3$, since Y's domain at that point has no value greater than 3; X has no neighbour other than Y, so nothing is re-queued. Revising (Y, X) removes $Y = 1$, since X's domain has no value less than 1. Y changed, so every arc that *points at* Y from a neighbour other than X must be re-checked: that is (Z, Y), which goes to the back of the queue. Revising (Y, Z) removes $Y = 3$ (no $z > 3$ in Z), leaving $Y = \{2\}$; Y changed again, so (X, Y) is re-queued. Now (Z, Y) comes up, and against $Y = \{2\}$ it removes both $Z = 1$ and $Z = 2$ (neither has a $y < z$ support), leaving $Z = \{3\}$. The duplicate (Z, Y) entry is popped next and finds nothing to do. Finally, the re-queued (X, Y) is revised against $Y = \{2\}$ and removes $X = 2$ (no $y > 2$ left), leaving $X = \{1\}$. The worklist is now empty: the fixpoint is $X = \{1\}, Y = \{2\}, Z = \{3\}$ – for this particular small chain, arc consistency alone reached the *unique* solution, with no search at all. Note the direction of the re-queue rule: when (X_i, X_j) shrinks X_i, the arcs (X_k, X_i) for the other neighbours X_k are re-queued, never (X_i, X_k) – a smaller X_i can only remove supports *from* its neighbours. Figure 2.1 shows this same fixpoint as a before/after domain table.

A one-sentence forward pointer, not developed further here: chain- and tree-shaped CSPs like this one are a case where arc consistency alone always reaches a unique solution without search – Week 5 makes that precise.

The trace above is independently checkable against real code, not just a hand argument.

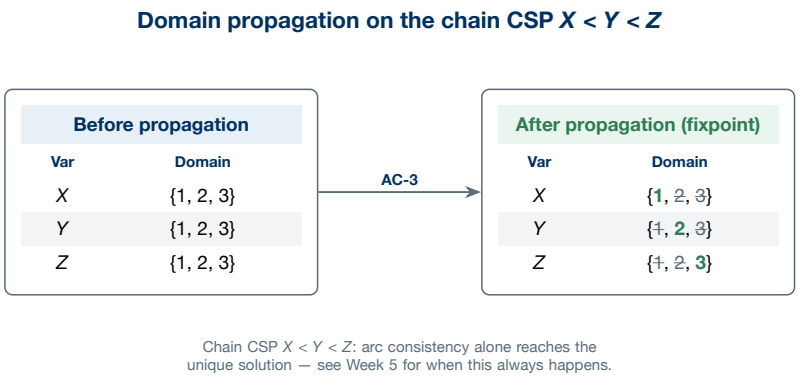


Figure 2.1: The chain CSP $X < Y < Z$ before propagation and at the AC-3 fixpoint; struck-through values are the ones `Revise` removed.

```
domains = {"X": {1, 2, 3}, "Y": {1, 2, 3}, "Z": {1, 2, 3}}
arcs = [("X", "Y"), ("Y", "X"), ("Y", "Z"), ("Z", "Y")]
constraints = {
   ("X", "Y"): lambda x, y: x < y,
   ("Y", "X"): lambda y, x: x < y,
   ("Y", "Z"): lambda y, z: y < z,
   ("Z", "Y"): lambda z, y: y < z,
}
neighbours = {"X": ["Y"], "Y": ["X", "Z"], "Z": ["Y"]}
return domains, arcs, constraints, neighbours
```

builds exactly this chain CSP as plain Python data, with a domain per variable, a worklist of arcs, and one constraint function per arc:

```
changed = False
for value_i in set(domains[xi]):
   if not any(constraint(value_i, value_j) for value_j in domains[xj]):
      domains[xi].remove(value_i)
      changed = True
return changed
```

`revise` is the `Revise` step written directly from its definition: for every value still in X_i's domain, keep it only if some value in X_j's current domain supports it under the given constraint function, and report whether anything was removed. The worklist loop that calls it, re-queueing neighbours exactly as the trace above describes, is equally direct:

```
worklist = deque(arcs)
while worklist:
   xi, xj = worklist.popleft()
   if revise(domains, xi, xj, constraints[(xi, xj)]):
      if not domains[xi]:
         return False
      for xk in neighbours[xi]:
         if xk != xj:
            worklist.append((xk, xi))
return True
```

Running this on the chain CSP produces:

```
print(f"Domains before AC-3: {format_domains(domains)}")
consistent = ac3(domains, arcs, constraints, neighbours)
print(f"Arc-consistent: {consistent}")
print(f"Domains after AC-3 (fixpoint): {format_domains(domains)}")
```

Expected Output

```
Domains before AC-3: X=[1, 2, 3], Y=[1, 2, 3], Z=[1, 2, 3]
Arc-consistent: True
Domains after AC-3 (fixpoint): X=[1], Y=[2], Z=[3]
```

This is the real, captured output of the code above, and it matches the hand trace exactly: $X = \{1\}$, $Y = \{2\}$, $Z = \{3\}$ at the fixpoint. When a hand trace and a real program disagree, the program is not automatically right and the hand trace automatically wrong – but in this book, every hand-worked trace is checked against a real run before it is written down, precisely so the two never contradict each other in what you read.

2.2.3 Termination and worst-case complexity

The state AC-3 reaches once its worklist finally empties – the state where no further `Revise` call anywhere in the problem would change any domain – is worth a name of its own, because the same shape of "keep revising until nothing changes" recurs elsewhere in this course under different arithmetic. This unit calls that state a **fixpoint**: the chain-CSP trace above reached exactly this state at $X = \{1\}$, $Y = \{2\}$, $Z = \{3\}$, and $A \leq B, B \leq C$'s trace below reaches one too, even though its domains do not shrink at all.

fixpoint: The state reached once no further Revise call changes any domain – propagation has nothing more to remove.

Definition: Fixpoint

The state reached once no further `Revise` call changes any domain – propagation has nothing more to remove. Weeks 7 and 10 reuse this same word for a different kind of repeated-update process (LP pivoting, dual updates), in different arithmetic; the shape – keep revising until nothing changes – is the part that carries over.

AC-3 always terminates, reaching a fixpoint rather than looping forever. Every call to `Revise` either removes at least one value from a domain or changes nothing at all; a domain cannot shrink past empty (once it is empty, the CSP is already proven infeasible and there is nothing further to remove); so the worklist can only be refilled finitely many times before no further removal is possible anywhere, and the loop stops. This is a one-sentence argument, but it is the entire reason AC-3 is guaranteed to halt.

Complexity (big-O)

The classic worst-case time bound for AC-3 on a binary CSP is $O(e \cdot d^3)$, where e is the number of constraints (arcs) and d is the largest domain size – each arc can be revised up to d times (once per value it might lose), and each individual `Revise` call costs up to $O(d^2)$ to check every value of X_i against every value of X_j. This bound is the standard textbook derivation (see, for example, Apt's or Dechter's constraint-processing textbook for the full argument) and is stated and used here, not re-derived from scratch.

3 • Participation **Now You Try**

Trace AC-3 by hand on the CSP $A \leq B$, $B \leq C$, all three domains $\{1, 2\}$. List the arcs processed, in order, with the domain change (if any) after each, and give the domains at the fixpoint.

Feedback. (A, B): every value of A has a support in B under $\leq$ ($A = 1$ supported by $B \in \{1, 2\}$; $A = 2$ supported by $B = 2$) – no change. (B, A): $B = 1$ needs some $A \leq 1$, supported by $A = 1$ – no change; $B = 2$ needs some $A \leq 2$, supported – no change. (B, C): symmetric to (A, B) – no change. (C, B): symmetric to (B, A) – no change. The worklist empties with **no domain reduced at all**: $A = \{1, 2\}$, $B = \{1, 2\}$, $C = \{1, 2\}$. This is a deliberately different outcome from the $X < Y < Z$ trace above ($\leq$ versus $<$): arc consistency does not always prune anything, and a CSP can be fully arc-consistent while still admitting several – here, many – solutions. Arc consistency is a *necessary* filtering step, not a solving procedure by itself, except in special cases like the strict-inequality chain worked earlier in this unit.

Check Yourself 2.2

State AC-3's worst-case time complexity in terms of the number of binary constraints e and the largest domain size d, and explain in one sentence why the algorithm is guaranteed to terminate.

5 • Follow-through **Transfer — Where Else?**

The worklist/fixpoint pattern you just traced by hand is not specific to CSPs. Week 10's shadow-price and reduced-cost updates in LP sensitivity analysis and Week 15's unsat-core extraction both reuse the same "keep revising until nothing changes" shape, in different notation – recognizing the pattern here means recognizing it again later instead of re-learning it from scratch.

2.3 Global Constraints: What They Propagate

1 • Pre-instruction **Why It Matters**

The `AddAllDifferent` call you already wrote in Week 1 was doing more work than a same-length line of code has any right to do – it was running a graph-matching algorithm every time it filtered a domain. The four constraints this unit adds are the same idea in four different disguises: each one hides real, nontrivial reasoning behind a one-line call.

1 • Pre-instruction **Prior Knowledge**

Unit 1 defined generalized arc consistency and worked through its pigeonhole toy on `AllDifferent(A, B, C)`; this unit revisits that exact example by name. Week 1 Unit 2's idea that auxiliary variables exist to make a constraint "easier to state" also returns here – `Element`'s index/target pair is exactly that kind of bookkeeping device, formalized as a constraint in its own right.

global constraint: A constraint stated over an entire group of variables at once, whose filtering can use information a set of smaller (e.g. binary) constraints over the same variables cannot see jointly.

A **global constraint** is a constraint stated over an entire group of variables at once, whose filtering can use information a pile of smaller (typically binary) constraints over the same variables cannot see jointly – exactly the gap Unit 1's pigeonhole example demonstrated for `AllDifferent`. This unit works through five global constraints that recur throughout this course: `AllDifferent`, `Element`, `Table`, `Circuit` and `Cumulative`. Figure 2.2 previews all five before their subsections are read one by one.

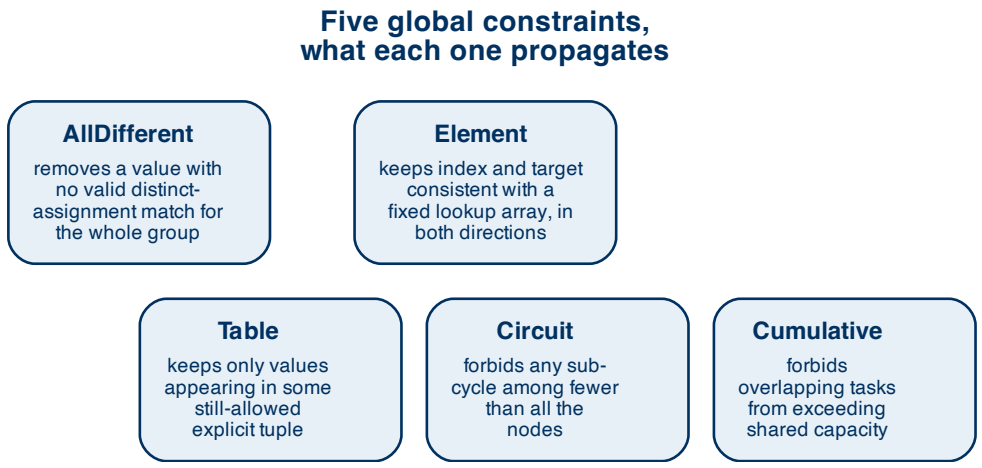


Figure 2.2: Five global constraints, what each one propagates – an at-a-glance overview before the detailed treatment below.

All five share the same shape as Unit 1's consistency ladder: a global constraint is a single filtering rule over a whole group of variables that can see relationships a pile of small binary constraints cannot, because the small constraints never get to look at the group as a whole. Two of the five – `Circuit` and `Cumulative` – appear here only as tiny, self-contained propagation demonstrations, deliberately far smaller than a real routing or scheduling instance; full treatments follow in Week 14 and Week 11 respectively.

2.3.1 AllDifferent: filtering by matching, not by pairs

Régin's algorithm: The matching-based filtering algorithm that achieves GAC on an AllDifferent constraint: a value survives only if it participates in some perfect matching assigning every variable in the constraint a distinct value.

Unit 1's pigeonhole toy showed *what* `AllDifferent` catches that a pairwise decomposition misses; this subsection names *how*. `model.AddAllDifferent(…)` achieves true GAC using **Régin's algorithm**: build a bipartite graph with one side holding the variables and the other holding the values still in their domains, connect a variable to every value still available to it, and keep a value in a variable's domain only if it takes part in *some* perfect matching of that graph – an assignment of every variable to a distinct value. A value with no such matching can never appear in any solution, and Régin's algorithm finds and removes exactly those values, achieving the same result the pigeonhole example reached by hand, on CSPs far too large to check that way manually.

> **Tip**
>
> This unit states what Régin's algorithm guarantees – true GAC on `AllDifferent` – without deriving matching theory from scratch; the Rossi–van Beek–Walsh handbook chapter on global constraints is the standard reference for the full

derivation, for anyone who wants it.

2.3.2 Element: a lookup that propagates in both directions

Element constraint: The global constraint `target == array[index]` for a fixed array and decision variables `index`/`target`, propagating in both directions between the two.

The `Element` constraint formalizes a pattern you have already used informally: looking something up in a fixed table using a variable index. An **Element constraint** forces `target == array[index]` for a *fixed* array and *decision* variables `index` and `target`. What makes `Element` worth a name of its own, rather than just a formula, is that it propagates in *both* directions: narrowing `index`'s domain narrows `target`'s domain to the set of array entries the remaining indices could point to, and narrowing `target`'s domain – through some other constraint entirely – narrows `index`'s domain back down to only the indices whose array entry is still allowed.

Analogy

Think of a numbered menu board: you might not know exactly which item number you will order yet, but if you already know the price must be at least a certain amount, you can already rule out every item number whose listed price is too low – the lookup narrows in both directions, not just from item number to price.

```python
model = cp_model.CpModel()
index = model.NewIntVar(0, len(prices) - 1, "index")
target = model.NewIntVar(min(prices), max(prices), "target")
model.AddElement(index, prices, target)
before = _enumerate_values(model, index)

model.Add(target >= 20)
after = _enumerate_values(model, index)

print(f"Element demo, menu prices={prices}")
print(f"index domain before target >= 20: {before}")
print(f"index domain after target >= 20: {after}")
```

A fixed menu-price array `[10, 25, 40, 15]` backs `index` (which item was ordered) and `target` (its price) with a single `AddElement` call. Before any further constraint, `index` can still be any of its four values. Adding `target >= 20` propagates *backward* through the same constraint, removing every index whose price is below 20:

Expected Output

```
Element demo, menu prices=[10, 25, 40, 15]
index domain before target >= 20: [0, 1, 2, 3]
index domain after target >= 20: [1, 2]
```

Indices 0 and 3 (prices 10 and 15) are gone, leaving only $\{1, 2\}$ (prices 25 and 40) – exactly the two items priced at least 20 – and this happened without ever fixing `index` directly. This is the real captured output of the code above.

2.3.3 Table: an explicit list of allowed combinations

Table constraint: A global constraint whose allowed combinations are given as an explicit list of tuples rather than a formula.

Not every relation reduces to a clean algebraic formula. A **Table constraint** states the relation between a group of variables as an explicit list of jointly-allowed value

tuples, rather than a formula like `!=` or `<`; it propagates by keeping, for each variable, only the values that still appear in at least one allowed tuple compatible with every other variable's current domain.

Analogy

An explicit room/time compatibility chart from a facilities office works the same way: rather than writing one general rule, the chart just lists every allowed pairing directly – useful exactly when the allowed combinations do not reduce to a clean formula.

```
model = cp_model.CpModel()
color1 = model.NewIntVar(0, 2, "color1")
color2 = model.NewIntVar(0, 2, "color2")
model.AddAllowedAssignments([color1, color2], allowed)
model.Add(color1 == 0)
remaining = _enumerate_values(model, color2)

print(f"Table demo, allowed (color1, color2) pairs: {allowed}")
print(f"color1 fixed to {colours[0]}; color2 narrows to "
    f"{[colours[v] for v in remaining]}")
```

Two variables `color1`, `color2`, domain $\{R, G, B\}$ each, under a `Table` constraint listing exactly the 6 allowed `(color1, color2)` pairs where `color1 != color2`. Fixing `color1 == 0` (that is, R) and re-checking `color2`'s domain gives:

Expected Output

```
Table demo, allowed (color1, color2) pairs: [(0, 1), (0, 2), (1, 0), (1,
↪  2),
(2, 0), (2, 1)]
color1 fixed to R; color2 narrows to ['G', 'B']
```

In this particular case the Table constraint filters identically to a plain `color1 != color2` constraint, because its allowed-tuple list was built to contain exactly the pairs `!=` already allows. That raises an obvious question: if the two filter the same here, what is Table actually for? The answer is any situation where the allowed combinations do *not* reduce to a clean symbolic rule – an irregular compatibility chart from a facilities office, for instance, where some same-looking pairs are allowed and others are not for reasons no single inequality captures. `!=` can only ever express "these two must differ"; `Table` can express any relation at all, including ones with no algebraic shorthand – which is the actual point of having it available.

2.3.4 Circuit: forcing one connected loop, not several

Circuit constraint: The global constraint forcing a chosen set of arcs to form a single Hamiltonian cycle over all nodes, rather than several disjoint sub-cycles.

The **Circuit constraint** takes a set of candidate arcs between nodes and forces the chosen ones to form a single Hamiltonian cycle – every node visited exactly once, in one connected loop that returns to its start – rather than a disjoint union of several smaller cycles that happen to cover all the nodes between them.

Analogy

Think of a single courier's delivery route: every stop must be visited exactly once and the route must return to where it started, as one connected loop, not

several separate mini-loops covering different stops.

`Circuit` propagates by forbidding any subset of the chosen arcs that would close a sub-cycle among fewer than all the nodes before the full assignment is complete – for example, a loop through only 3 of 6 nodes is ruled out the moment enough of its arcs are chosen to make that sub-loop unavoidable, long before the remaining nodes are even considered. This rules out large classes of partial assignments early, well before a full route is built.

Caution

This unit's `Circuit` example is a deliberately tiny, 4-node toy graph – a self-contained propagation demonstration, not a routing model. Full vehicle-routing content, with real distances, capacities and multiple vehicles, is Week 14's job; do not expect scheduling-scale machinery from this small a demo.

```
model = cp_model.CpModel()
arcs = [
   (i, j, model.NewBoolVar(f"arc_{i}_{j}"))
   for i in range(n)
   for j in range(n)
   if i != j
]
model.AddCircuit(arcs)

solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
solver.Solve(model)
chosen = {(i, j) for i, j, lit in arcs if solver.Value(lit)}
cycle = _extract_cycle(chosen, n)
print(f"Circuit demo ({n} nodes): visiting order {cycle} -> back to {cycle
    [0]}")
```

Solving this 4-node toy with `model.AddCircuit(arcs)` produces one valid Hamiltonian cycle:

Expected Output

```
Circuit demo (4 nodes): visiting order [0, 2, 1, 3] -> back to 0
```

Every one of the 4 nodes appears exactly once in this order, and the route returns to node 0 at the end – a single connected loop, exactly what `Circuit` requires, read directly from a real captured run.

2.3.5 Cumulative: a shared resource's capacity over time

Cumulative constraint: The global constraint forbidding any set of time-overlapping tasks from exceeding a shared resource's capacity at any instant.

The **Cumulative constraint** models tasks that each have a start time, a fixed duration, and a demand on one shared resource, and it forbids any set of overlapping tasks whose demands sum past the resource's capacity at any instant during their overlap.

Analogy

A shared lab room's simultaneous-occupancy cap works the same way: several experiments, each with its own start time and duration and headcount, all draw on the same room's capacity while their times overlap, and the room's capacity can never be exceeded at any overlapping moment.

Caution

As with `Circuit` above, this unit's `Cumulative` example is a small, self-contained propagation demonstration – 3 tasks on one resource – not a full scheduling model. Interval variables, `NoOverlap`, and makespan objectives are Week 11's subject.

```
model = cp_model.CpModel()
starts = [model.NewIntVar(0, horizon, f"start_{t}") for t in range(3)]
intervals = [
   model.NewFixedSizeIntervalVar(starts[t], durations[t], f"task_{t}")
   for t in range(3)
]
model.AddCumulative(intervals, demands, capacity)

solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
solver.Solve(model)
result = [solver.Value(s) for s in starts]
print(f"Cumulative demo, durations={durations}, demands={demands}, "
    f"capacity={capacity}: start times = {result}")
```

Three tasks, durations `[3, 2, 2]` and demands `[2, 3, 2]` on a resource of capacity 4, built as fixed-size interval variables and passed to `AddCumulative`, solve to:

Expected Output

```
Cumulative demo, durations=[3, 2, 2], demands=[2, 3, 2], capacity=4:
start times = [2, 0, 2]
```

Checking this by hand confirms the constraint held: task 1 (demand 3) occupies $[0, 2)$ alone; tasks 0 and 2 (demands 2 and 2) both occupy $[2, 4)$, together demanding $2 + 2 = 4$, exactly at – never past – the resource's capacity of 4. `Cumulative` ruled out every start-time combination that would have pushed a moment's total demand above 4 before any of these three start times was even fixed.

2.3.6 Why five different names for one shared idea

Put the five together and a single pattern emerges: every one of them hides a non-trivial algorithm – a matching, a bidirectional table lookup, a tuple filter, a sub-cycle check, a resource-capacity check – behind a one-line model call, and every one of them can see something a pile of smaller, per-pair constraints over the same variables cannot see jointly. That is precisely Unit 1's GAC idea, applied five times to five different shapes of relation.

3 • Participation **Now You Try**

For the two variables `color1`, `color2`, domain $\{R, G, B\}$ each: (a) a Table constraint listing the 6 allowed `(color1, color2)` pairs where `color1 != color2`, and (b) a plain `color1 != color2` constraint. Do they filter the same values here? Then describe one situation, different from this one, where Table and `!=` would *not* agree.

Feedback. Yes, here they filter identically, because the Table's allowed-tuple list was built to contain exactly the pairs `!=` already allows. They would disagree whenever the allowed combinations do not reduce to a clean symbolic rule – for example, an explicit room/time compatibility chart from a facilities office, where some same-looking pairs are allowed and others are not for reasons no single inequality captures. That is precisely when Table earns its place over a symbolic constraint: it can express relations `!=` simply cannot.

Check Yourself 2.3

In one or two sentences, explain why decomposing `AllDifferent(A,B,C)` into the three pairwise constraints $A \neq B$, $A \neq C$, $B \neq C$ can fail to detect an infeasible CSP that the global constraint's GAC filtering (Régin's algorithm) detects immediately.

5 • Follow-through **Transfer — Where Else?**

You will meet these five constraints again by name, not just by concept: `AllDifferent` returns in Week 6's three-solver comparison, `Cumulative` becomes the backbone of Week 11's resource-constrained scheduling, `Circuit` becomes Week 14's routing models, and `Table` and soft variants of these ideas reappear in Week 8's over-constrained problems and Week 12's timetabling. Recognizing "this is just a Table constraint" or "this is just Cumulative" when reading a new problem statement is a large part of what modeling skill – Week 1 Unit 2 – turns out to mean in practice.

2.4 Measuring Propagation Strength: Weak vs. Strong Models

1 • Pre-instruction **Why It Matters**

Unit 1 argued, on paper, that a global constraint's filtering can catch what a pile of pairwise constraints misses. Arguments on paper are convincing; a solver's own printed statistics, from a model you built and ran yourself, are proof – this unit is where you go get that proof.

1 • Pre-instruction **Prior Knowledge**

Week 1's `nqueens.py` (the strong model, reused unchanged here) and this week's Unit 1 pigeonhole argument plus Unit 3's `AllDifferent`/Régin explana-

tion give you everything you need to *expect* the strong model to need less search work than a pairwise decomposition of the same relation. This unit does not introduce a new problem – it instruments a problem you have already solved.

propagation strength: How much infeasible value space a set of constraints removes before or during search, for the same underlying problem.

Two CP-SAT models of the *same* 8-queens instance make this unit's central idea, **propagation strength**, measurable rather than merely argued on paper. The first model is Week 1's `nqueens.py`, unchanged: one global `AllDifferent` over the columns, and one more over each diagonal transform. Call this the *strong* model. This unit builds a second model of the identical N-queens relation next, then runs both and reads what CP-SAT itself reports about the search each one needed.

A second model expresses that identical relation as a **decomposition**: a plain `!=` constraint for every pair of columns and every pair of diagonal values, using the exact same variables and diagonal transforms. Call this the *weak* model. The *only* thing that differs between the two models is how the "no two share a value" relation is stated – one global constraint per group versus $\binom{n}{2}$ pairwise constraints per group.

decomposition: Expressing a relation intended for a global constraint as an equivalent set of smaller (typically binary) constraints instead.

> **Caution**
>
> Map coloring is deliberately *not* used for this comparison. Map coloring's adjacency relation only requires *adjacent* regions to differ, not every region from every other region – replacing its pairwise `!=` edges with a single `AllDifferent` over all regions would change which assignments count as solutions, not just the propagation strength of an equivalent model. This exact question returns as this unit's Check Yourself item below.

```
queens = [model.NewIntVar(0, n - 1, f"queen_{row}") for row in range(n)]
return queens
```

Both models share this one variable-declaration function, carried over from Week 1 unchanged: one integer variable per row, domain `0` to `n-1`. The strong model's constraint layer is exactly Week 1's three `AddAllDifferent` calls:

```
model.AddAllDifferent(queens)
model.AddAllDifferent([queens[i] + i for i in range(n)])
model.AddAllDifferent([queens[i] - i for i in range(n)])
```

The weak model's constraint layer states the identical relation with nested-loop pairwise `!=` constraints instead – no `AddAllDifferent` anywhere:

```
for i in range(n):
   for j in range(i + 1, n):
      model.Add(queens[i] != queens[j])
      model.Add(queens[i] + i != queens[j] + j)
      model.Add(queens[i] - i != queens[j] - j)
```

2.4.1 Reading solver statistics, not just the answer

After `solver.Solve(model)`, CP-SAT reports more than a status and an assignment. `solver.NumBranches()`, `solver.NumConflicts()` and `solver.WallTime()` report how much work the search actually did to reach a solution – the empirical stand-in for "how much did propagation prune before or during search," since a model whose propagation removes more dead values needs fewer branches to find or rule out an assignment.

```
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
return {
   "status": solver.StatusName(status),
   "branches": solver.NumBranches(),
   "conflicts": solver.NumConflicts(),
   "wall_time": solver.WallTime(),
}
```

`collect_solver_stats` solves one model and packages exactly those three statistics (plus the solve status) into a small record. Calling it once per model and printing both side by side is the entire comparison:

```
strong_model, _ = build_strong_model(n)
weak_model, _ = build_weak_model(n)
stats = {
   "strong (AllDifferent)": collect_solver_stats(strong_model),
   "weak (pairwise !=)": collect_solver_stats(weak_model),
}
print(f"N={n} propagation-strength comparison")
for label, s in stats.items():
   print(
      f"{label}: status={s['status']} branches={s['branches']} "
      f"conflicts={s['conflicts']} wall_time={s['wall_time']:.4f}s"
   )
return stats
```

Running this for $N = 8$ produces the following real, captured output:

Expected Output

```
N=8 propagation-strength comparison
strong (AllDifferent): status=OPTIMAL branches=475 conflicts=9
↪  wall_time=0.0021s
weak (pairwise !=): status=OPTIMAL branches=518 conflicts=0
↪  wall_time=0.0027s
```

On branches and wall time, this run points the direction Unit 1 and Unit 3's argument predicts: the strong model needed fewer branches (475 against 518) and less wall-clock time (0.0021s against 0.0027s) to reach an optimal solution. On conflicts, though, the strong model actually reported *more* (9 against 0) – the opposite of what a first glance at the argument might suggest.

Caution

Do not paper over that last number. This book's rule is to report the real numbers from the real run, including a result that does not fit the expected pattern perfectly. A defensible reading, offered here as an observation rather than a proven claim: `NumConflicts` counts search dead-ends encountered along the way, and a model that reaches its answer in fewer total branches can still have a higher *proportion* of those branches end in a conflict without that making its overall search more expensive – branches and wall time, which both agree here, are the metrics that most directly track total search effort. Whether that reading holds up in general is exactly the kind of question a larger instance can help answer, which is what the $N = 20$ run below is for.

CP-SAT's own presolve is itself quite good at recognizing certain pairwise decompositions and tightening them before search even begins, which is part of why the gap at $N = 8$ is modest rather than dramatic – this does not make Unit 1 and Unit 3's argument wrong, it makes CP-SAT a strong baseline to be measuring against. Running the same comparison at a noticeably larger $N = 20$, still pure satisfaction, gives:

Expected Output

```
N=20 propagation-strength comparison
strong (AllDifferent): status=OPTIMAL branches=3312 conflicts=51
↪  wall_time=0.0090s
weak (pairwise !=): status=OPTIMAL branches=3940 conflicts=26
↪  wall_time=0.0201s
```

The same pattern repeats at this larger size: the strong model uses fewer branches (3312 against 3940) and, this time, noticeably less wall-clock time – more than twice as fast (0.0090s against 0.0201s) – while again reporting more raw conflicts (51 against 26). Taken together, the two runs support the argument on the metrics that most directly reflect total search cost (branches, wall time), while the conflict count is a genuine, reportable exception to a naive one-directional prediction, not a result to hide or explain away with an unproven claim.

Common Pitfall

A frequent mistake reading solver statistics is assuming a single metric must always move in the "expected" direction, and either discarding a run that does not or quietly not reporting it. Every statistic in this book comes from a real, captured run, never a guess at what "should" happen – the honest, useful finding from this unit's real run is that branches and wall time agree with the theoretical argument while conflicts do not, and that nuance is itself informative, not noise to be edited away.

Figure 2.3 turns the $N = 8$ numbers above into a single chart, giving you a visual companion to the same real numbers just discussed in prose.

3 • Participation **Now You Try**

Run both `nqueens_models.py`'s strong (`AllDifferent`) model and its pairwise-`!=` weak model for $N = 8$, record `NumBranches()`, `NumConflicts()` and `WallTime()` for each from your own terminal output, and state which model's statistics indicate stronger propagation on this run.

Feedback. There is no single fixed numeric answer to copy – read your own `ciktilar/nqueens-propagation-stats.txt`. The expected *direction*, per Unit 1 and Unit 3's argument, is that the `AllDifferent` model needs fewer branches and less wall time than the pairwise-`!=` decomposition, though CP-SAT's presolve can narrow this gap at $N = 8$, and even a metric moving the "wrong" way (as `NumConflicts` did in the captured run above) is a legitimate observation to report, not an error to hide – and a cue to try a larger N, exactly as this unit's own $N = 20$ run did.

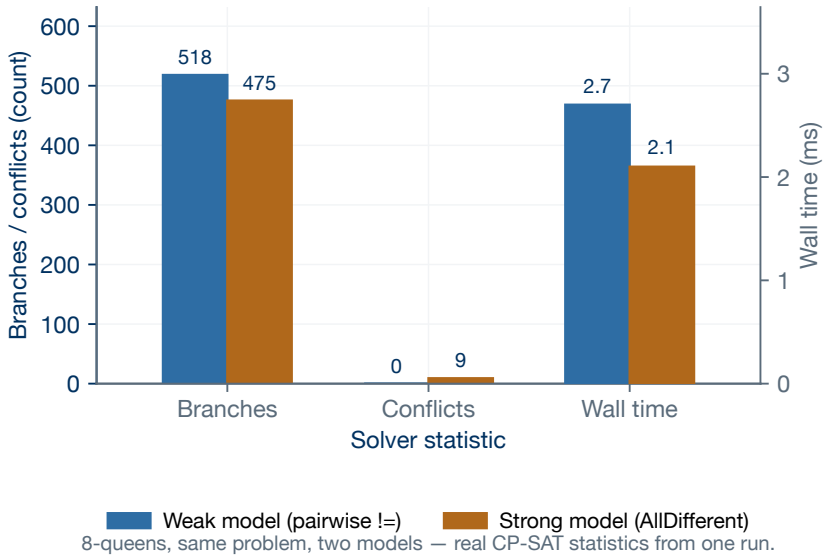


Figure 2.3: Weak (pairwise `!=`) versus strong (`AllDifferent`) 8-queens models: real `NumBranches`, `NumConflicts` and `WallTime` from one captured run.

Check Yourself 2.4

A student proposes simplifying Week 1's map-coloring model by replacing every pairwise `!=` adjacency constraint with a single `AllDifferent` over *all* regions at once. Is this a valid, meaning-preserving change? Why or why not?

5 • Follow-through **Transfer — Where Else?**

"Same problem, two models, compare the real solver statistics" is not a one-off exercise. Week 6 runs the exact same idea across three different solver back-ends (Gecode, Gurobi and HiGHS via MiniZinc), Week 7 repeats it between CP-SAT and a Gurobi MIP formulation of the same problem, and Week 15 turns it into a formal benchmarking methodology with statistical comparison across many instances. The two small `.txt` files you produced this week are your first entries in a habit this course keeps asking for.

Chapter Summary

Summary

This chapter opened the box last week left closed. Unit 1 defined three increasingly strong levels of local consistency – node consistency (a variable checked against its own unary rules alone), (binary) arc consistency (a variable checked against one neighbour's current domain, via the idea of support), and generalized arc consistency (a variable checked against an entire constraint's group of variables jointly) – and used a three-variable, two-value pigeonhole example to show that GAC can catch an infeasibility that a purely pairwise check misses entirely. Unit 2 turned arc consistency from a static definition into an algorithm: AC-3 maintains a worklist of arcs, repeatedly applies a `Revise` step

that removes unsupported values, re-queues affected neighbours whenever a domain changes, and always terminates at a fixpoint – proven by a real, independently checkable code trace on the chain CSP $X < Y < Z$ – with a textbook worst-case bound of $O(e \cdot d^3)$. Unit 3 named the algorithm behind `AllDifferent` (Régin's matching-based filtering) and worked through four more global constraints – `Element`, `Table`, `Circuit` and `Cumulative` – each hiding a nontrivial filtering rule behind a one-line model call, demonstrated on small toy instances with real captured output. Unit 4 closed the chapter by building a strong (`AllDifferent`) and a weak (pairwise `!=`) CP-SAT model of the identical 8-queens relation and reading CP-SAT's own `NumBranches`, `NumConflicts` and `WallTime` statistics from real runs at $N = 8$ and $N = 20$ – branches and wall time favoured the strong model at both sizes, while conflicts, honestly, did not, an instructive reminder that empirical propagation-strength evidence is read from whatever a real run actually reports, not from what an argument alone predicts. Next week takes this exact pair of N-queens models and opens the search box these two weeks deliberately left closed: you will watch a solver's search tree grow, branch by branch, and compare variable- and value-ordering heuristics on the same models you have now built two versions of.

Check Yourself Answers

Question (Unit 1 – Preferred Campus and arc consistency). Not automatically. Node consistency only checks each field's own unary rule in isolation – removing Downtown from "Preferred Campus" is a purely single-field filtering step and says nothing yet about the second field. Arc consistency additionally requires an explicit further check: for every value still remaining in one field's domain, does the other field still have some *different* value available? With {North, South} left in both fields after removing Downtown, that additional check does happen to hold here – each remaining value in one field still has a supporting, different value in the other – but it is a genuinely separate check, not something that follows automatically from removing Downtown alone; a case with only one value left in both fields would fail this same check even after identical unary filtering.

Question (Unit 2 – AC-3's complexity and termination). AC-3's textbook worst-case time bound is $O(e \cdot d^3)$, where e is the number of binary constraints (arcs) and d is the largest domain size. It is guaranteed to terminate because every `Revise` call either removes at least one domain value or leaves every domain unchanged, and no domain can shrink past empty, so the worklist can only be refilled finitely many times before no further change anywhere is possible – at which point the loop stops at a fixpoint.

Question (Unit 3 – why pairwise decomposition can miss what GAC catches). Pairwise arc consistency on $A \neq B$, $A \neq C$, $B \neq C$ only ever checks, one pair at a time, whether each remaining value has *some* compatible partner in the other variable of that one pair; it never checks whether a single joint assignment satisfying every pair at once exists across the whole group of three variables together. The classic three-variables/two-values case (Unit 1's pigeonhole example, reused for `AllDifferent` by Régin's algorithm in Unit 3) is arc-consistent on every pair individually – every value always has *some* supporting partner in each one other

variable – yet by the pigeonhole principle it has no valid all-different assignment at all across all three simultaneously. Only a whole-group consistency check, which is exactly what GAC on the global constraint performs, reveals that.

Question (Unit 4 – would a single AllDifferent be a valid simplification of map coloring?). No. `AllDifferent` over every region at once would force *every* region to receive a colour distinct from *every* other region, including pairs of regions that are not adjacent and therefore never needed to differ in the first place – a strictly stronger, and wrong, restriction. Map coloring's actual requirement is only that *adjacent* regions differ; replacing map coloring's pairwise `!=` adjacency edges with one global `AllDifferent` would change which assignments count as solutions, not merely the propagation strength of an equivalent model. Replacing a pairwise decomposition with a single global constraint is only a valid, meaning-preserving move when the intended relation genuinely is "everyone in this group must differ from everyone else," the way it is for N-queens' column and diagonal values in Unit 4's comparison – not whenever a shortcut merely looks similar on the surface.